basic_protocols 技术文档

1. 技术方案原理

1.1 模块定位

basic_protocols 是当前工程中最底层的安全计算协议模块，目标是验证和封装二方秘密分享场景下的基础算子。它不直接面向完整神经网络推理，而是为上层线性层、非线性层、混合近似层提供可复用的安全乘法、三输入乘法、符号门生成和 ReLU 组合能力。

本模块的核心协议包括：

协议	英文含义	输入	输出	目标函数
SMP	Secure Multiplication Protocol	$\langle x\rangle_0/\langle x\rangle_1$, $\langle y\rangle_0/\langle y\rangle_1$	$\langle z\rangle_0/\langle z\rangle_1$	z = x * y
3SMP	Three-input Secure Multiplication Protocol	$\langle x\rangle_i, \langle y\rangle_i, \langle w\rangle_i$	$\langle z\rangle_i$	z = x * y * w
ODReLU	Oblivious DReLU	$\langle x\rangle_0/\langle x\rangle_1$	$\langle g\rangle_0/\langle g\rangle_1$	g = 1[x >= 0]
ReLU	Secure ReLU	$\langle x\rangle_0/\langle x\rangle_1$	$\langle z\rangle_0/\langle z\rangle_1$	z = max(x, 0)

协议均以批量向量形式实现。一个函数调用通常处理 count 或 vector_size 个元素，所有元素共享同一套协议结构，但每个元素使用独立随机量。这样设计可以减少函数调用、线程创建和网络同步开销，更接近真实模型推理时的批处理需求。

1.2 系统参与方

模块采用二方安全计算加可信执行环境辅助的结构。参与方可以抽象为：

角色	主要职责	可见数据
Party0	持有第 0 份秘密分享，发起或响应协议，执行本地补偿计算	本地输入分享、本地随机量、网络消息、本地输出分享
Party1	持有第 1 份秘密分享，执行对称协议逻辑	本地输入分享、本地随机量、网络消息、本地输出分享
Enclave	生成或保存协议随机性，计算被掩码的辅助	ECALL 输入、TEE 内部随机状态、TEE 输

角色	主要职责	可见数据
	项	出辅助项
App	调度线程、网络通信、ECALL 调用、结果合成和计时	非可信侧缓冲区、协议公开量、输出分享

这里的 TEE 不是用来直接计算全部明文函数。它承担的是辅助随机化和补偿项生成。REE 负责主要工程调度，TEE 只处理协议需要保护的关键随机状态和部分代数补偿。

1.3 秘密分享与环上计算

模块默认采用二方加性秘密分享。对任意明文值 x，CSP0 和 CSP1 分别持有：

$$x = \langle x \rangle_0 + \langle x \rangle_1 \bmod L$$

其中 $\langle x \rangle_0$ 和 $\langle x \rangle_1$ 单独看都是随机形式，任意单方不能从自己的分享恢复 x。只有在测试或校验阶段，把两方输出分享相加，才得到重构结果。

当前实现以 uint64_t 为主要承载类型，因此自然工作在 64 位整数环：

$$L = 2^{64}$$

代码中的加法、减法和乘法都会依赖 uint64_t 溢出语义自动回到 $Z_{(2^{64})}$。在数学表达中可以统一写作：

$$(a + b) \bmod L$$
$$(a - b) \bmod L$$
$$(a * b) \bmod L$$

如果上层输入来自实数，需要先做定点编码。例如设缩放位数为 s：

$$x_{fixed} = \mathrm{floor}(x_{real} * 2^s) \bmod L$$

乘法会使缩放因子叠加。例如两个定点数相乘后理论缩放为 2^{2s}，若仍要回到原缩放，需要执行截断或重缩放。本模块函数签名中保留了 s、L 参数，说明协议接口可以承接定点缩放语义；但基础协议本身主要验证环上代数正确性，上层组合逻辑需要明确何时截断、何时保持原始乘积。

1.4 随机掩码的作用

基础协议的安全性来自两个层面的随机化。

第一层是输入分享本身的随机性。单个 CSP 只拿到 $\langle x \rangle_i$，不知道另一份分享，因此无法恢复 x。

第二层是协议在线阶段的掩码随机性。即使双方需要交换某些公开量，也不会直接交换 x 或 y，而是交换类似下面的被掩码值：

```
D = x + a
E = y + b
F = w + c
```

其中 a、b、c 是协议随机量。只要随机量不被同一观察者完整掌握，公开 D/E/F 不会直接暴露原始输入。

在代码中，SMP 使用 a、b、r_0 作为 TEE 侧随机状态；3SMP 使用 a、b、c、r_0 以及中间缓存 m_3；ODReLU 使用 a、b、r_0、r_1、v。这些随机量与网络消息共同构成协议的遮蔽层。

1.5 base 与 dualport

每类协议通常有两种工程形态：

形态	目标	主要手段
base	基础单向传输版本，强调协议代数和单通道执行路径	使用一个网络通道完成 masked message 传输，并在同一次协议调用内推进掩码计算、TEE/本地补偿
dualport	在 base 基础上进一步并行化双向通信	使用两个独立网络通道拆分通信负载，使每条通道只承担原 base 单通道约一半的通信量

base 和 dualport 不是两套不同的密码学协议，而是同一协议在工程实现上的两个阶段。base 是普通基础版本，重点是清楚表达协议本身的单通道、单向传输逻辑：一方沿固定连接发送 masked message，另一方接收后结合本地分享和 TEE 补偿生成输出分享。它不是为了强调“并行协议”，而是作为后续双通道优化的基准实现。 dualport 可以看作 base 的进一步演进版本。它保留基础协议的代数结构，同时把通信路径从单通道扩展为双通道，进一步降低双向通信阶段的阻塞。 在 base 版本中，网络数据主要沿单通道推进。若协议某一步需要双方同时交换 masked message，单通道上的发送和接收仍可能形成排队：一方发送大块数据时，另一方需要等待该方向的数据完成后才能继续推进后续通信。 dualport 的核心改进是把原来 base 单通道中的通信负载拆到两个端口上。每一方不再把全部消息压到同一条连接，而是通过两个独立通道同时推进收发；从通信量角度看，每条通道大约承担原 base 版本一半的数据量。这样总通信字节数不变，但通信时间可以由“两段串行传输”变成“两条通道并行传输”。在带宽和线程调度足够的情况下，双向通信阶段的等待时间理论上可以接近减半。 无论采用哪种工程形态，它们都必须满足同一个输出等式：

```
output₀ + output₁ = f(input) mod L
```

因此评估二者时，需要区分两个问题：一是代数正确性是否一致，二是调度是否减少了等待。base 更适合作为普通单通道基准；dualport 则是在 base 基础上继续优化网络路径，更关注双端口能否把原本集中在单通道上的通信量拆开并同时传输，尤其适合收发交替频繁、双方都需要主动发送中间量的协议阶段。

2. 逻辑推导

2.1 统一符号约定

设两方分享为：

```
x = x₀ + x₁ mod L
y = y₀ + y₁ mod L
w = w₀ + w₁ mod L
```

其中 $x_0/y_0/w_0$ 由 CSP0 持有，$x_1/y_1/w_1$ 由 CSP1 持有。协议目标不是让某一方得到 f(x)，而是让双方分别得到：z_0, z_1 并满足：

```
z = z₀ + z₁ mod L = f(x, y, w)
```

为了简化表达，以下推导先描述明文层面的代数关系，再说明如何转化为两方分享和 TEE 补偿。

2.2 SMP

2.2.1 代数展开

SMP 的目标是：

```
z = x * y mod L
```

引入随机掩码 a、b，公开被掩码后的值：

```
D = x + a
E = y + b
```

由此得到：

```
x = D - a
y = E - b
```

代入乘法：

```
xy = (D - a)(E - b)
   = D*E - D*b - E*a + a*b
```

这个式子揭示了 SMP 的核心结构：

项	含义	处理方式
D*E	公开掩码值的乘积	可以在 REE 侧根据公开量计算

项	含义	处理方式
-D*b	与 b 相关的补偿	由持有随机信息的一侧或 TEE 生成分享
-E*a	与 a 相关的补偿	由持有随机信息的一侧或 TEE 生成分享
a*b	掩码乘积补偿	作为随机补偿的一部分隐藏

最终双方分别构造 z_0、z_1，使：

```
z₀ + z₁
= D*E - D*b - E*a + a*b
= x*y
```

2.2.2 与代码变量的对应关系

在当前实现中，TEE 侧通过：

```
ecall_smp_gen_ab_r0_64(seed, count)
```

生成并保存：

```
a_array
b_array
r0_array
```

在线阶段的网络线程会构造类似：

```
mask_x = input_a + a
mask_y = input_b + b
```

也就是协议推导中的 D、E 或它们的单方局部版本。补偿线程使用 $r_0/r_1/r_2$ 等随机量把本地项拆成分享，避免某一方看到完整补偿。

TEE 的计算接口：

```
ecall_smp_tee1_computation_64(count, ..., x1_plus_r1, y1_plus_r2, result)
```

在 Enclave 中执行的核心形式为：

```
result = -b * (x₁ + r₁) - a * (y₁ + r₂) + r₀
```

这个式子对应 -D*b - E*a + a*b 中需要隐藏的部分。$r_0/r_1/r_2$ 的存在使补偿项不会以完整形式暴露在 REE 或网络中。

2.3 3SMP

2.3.1 代数展开

3SMP 的目标是：

```
z = x * y * w mod L
```

引入三个随机掩码：

```
D = x + a
E = y + b
F = w + c
```

于是：

```
x = D - a
y = E - b
w = F - c
```

展开：

```
xyw = (D - a)(E - b)(F - c)
    = D*E*F
      - a*E*F - b*D*F - c*D*E
      + a*b*F + a*c*E + b*c*D
      - a*b*c
```

3SMP 比 SMP 更复杂，因为它包含三类补偿：

类型	项	解释
公开主项	D*E*F	掩码公开值的三输入乘积
一阶负补偿	-aEF, -bDF, -cDE	移除一个真实输入被随机掩码替换带来的误差
二阶正补偿	abF, acE, bcD	修复展开中被重复扣除的二阶交叉项
三阶负补偿	-abc	移除三个掩码共同造成的残余项

如果直接用两次 SMP 计算 x*y*w，需要至少两轮乘法协议：先求 <xy>，再求 <xy*w>。3SMP 的目的在于把三输入乘法合并为一个更直接的协议，减少轮数和同步成本。

2.3.2 与代码变量的对应关系

TEE 初始化接口：

```
ecall_3smp_gen_abc_r0_64(seed, count)
```

生成：

```
a_array
b_array
c_array
r0_array
```

第一阶段 TEE 计算：

```
ecall_3smp_tee1_compute_m3_64(count, x1_plus_r1, y1_plus_r2, w1_plus_r3)
```

保存中间项：

```
m₃ = a*b*(w₁ + r₃)
   + a*c*(y₁ + r₂)
   + b*c*(x₁ + r₁)
```

第二阶段 TEE 计算：

```
ecall_3smp_tee1_compute_m4_64(count, U, V, W, final_result)
```

内部形成：

```
m₄ = -(a*U + b*V + c*W)
final_result = m₃ + m₄ + r₀
```

其中 U/V/W 可以理解为协议中已经组合出的公开或半公开乘积项，用来承载 EF/DF/DE 一类结构。m_3 负责二阶补偿，m_4 负责一阶负补偿，r_0 负责输出随机分享化。这样，3SMP 的复杂展开式被拆成两次 TEE 辅助计算和 REE 本地合成。

2.4 ODReLU

2.4.1 逻辑目标

ODReLU 的目标是生成符号门：

```
g = 1[x >= 0]
```

但协议不能直接公开 g，因为 g 本身泄露了输入 x 的符号。因此协议输出仍然是分享：

```
g = g₀ + g₁ mod L
```

后续 ReLU 使用 g 与 x 的秘密分享做安全乘法：

```
ReLU(x) = g * x
```

也就是说，ODReLU 是 ReLU 的比较子协议，SMP 是 ReLU 的乘法子协议。

2.4.2 随机仿射变换

ODReLU 使用随机仿射变换隐藏被比较的值。直观形式为：

```
z = a*x + b
```

其中 a 的正负由随机位 v 决定：

```
v = 0 => a > 0
v = 1 => a < 0
```

若 a > 0，z 与 x 的符号方向一致；若 a < 0，z 与 x 的符号方向相反。TEE 保存 v，并在返回结果时根据 v 修正门值，使外部无法从中间比较结果直接判断原始 x 的符号。

代码中随机生成逻辑大致为：

```
r_sign 决定 a 的符号
v₁, v₂ 生成不同的正数
abs_a = max(v₁, v₂)
abs_b = min(v₁, v₂)

if r_sign is even:
    a = abs_a
    b = abs_b
    v = 0
else:
    a = -abs_a
    b = -abs_b
    v = 1
```

这样 a/b 的绝对值和符号都带有随机性。r_0/r_1 用于隐藏 TEE step 的输入和输出，使 REE 看不到直接比较关系。

2.4.3 两步 TEE 逻辑

ODReLU 的 TEE 随机初始化接口为：

```
ecall_odrelu_gen_shared_randomness_final_uint64(seed, vector_size, xi)
```

生成并保存：

```
a_array
b_array
r0_array
```

```
r1_array
v_array
```

第一步 TEE 计算：

```
ecall_odrelu_tee1_step1_uint64(vector_size, x1_minus_r2, result)
```

内部计算：

```
result = a * (x₁ - r₂) - r₀
```

这一步把 CSP1 的输入分享与随机量结合，形成后续仿射比较需要的部分值。

第二步 TEE 计算：

```
ecall_odrelu_tee1_step2_uint64(vector_size, z_plus_r3, result)
```

内部根据 v 修正：

```
if v == 0:
    result = z_plus_r₃ + r₁
else:
    result = 1 - z_plus_r₃ + r₁
```

当 v == 0 时，随机仿射变换保持符号方向；当 v == 1 时，需要翻转门值。r_1 继续把返回值分享化，防止返回结果成为明文门值。

2.5 ReLU

2.5.1 组合推导

ReLU 的数学定义是：

```
ReLU(x) = max(x, 0)
```

引入符号门：

```
g = 1[x >= 0]
```

则：

```
ReLU(x) = g * x
```

分情况验证：

条件	g	g*x	ReLU 输出
x >= 0	1	x	x
x < 0	0	0	0

因此安全 ReLU 可以由两个基础协议组合得到：

```
<g> = ODReLU(<x>)
<z> = SMP(<g>, <x>)
```

组合过程中，g 不公开，x 不公开，乘积结果也不公开。双方只得到 ReLU(x) 的分享。

2.6 正确性归纳

四类协议的正确性可以统一表述为：

```
reconstruct(output_shares) = plaintext_function(reconstruct(input_shares))
```

具体为：

```
SMP:
z₀ + z₁ = x*y mod L

3SMP:
z₀ + z₁ = x*y*w mod L

ODReLU:
g₀ + g₁ = 1[x >= 0] mod L

ReLU:
z₀ + z₁ = max(x, 0) mod L
```

只要随机补偿项在两方输出中精确抵消，协议输出就满足目标函数。

3. 详细理论解释

3.1 基础安全模型

这一部分说明协议共同依赖的基础：分享形式、观察者视图、TEE 信任边界和随机量新鲜性。后续 SMP、3SMP、ODReLU、ReLU 都建立在这些约束之上。

3.1.1 为什么使用加性秘密分享

加性秘密分享有三个工程优势。

第一，它和环上整数计算天然兼容。`uint64_t` 的溢出语义正好对应 mod 2^{64}，加法、减法和乘法都可以直接用机器指令完成。

第二，它适合批量向量化。对长度为 n 的向量，只需逐元素满足：

```
x[i] = x₀[i] + x₁[i] mod L
```

协议可以把 n 个元素放在连续数组中，通过一次网络发送和一次批量 ECALL 处理。

第三，它便于组合。SMP 输出的仍然是加性分享，ODReLU 输出的也是加性分享，因此 ReLU 可以把 ODReLU 的输出直接作为 SMP 的输入，不需要改变分享格式。

3.1.2 安全性视图

可以从三个观察者视角理解协议安全性：

观察者	能看到什么	看不到什么	保护机制
单个 CSP	自己的输入分享、输出分享、网络收发缓冲、本地随机量	对方输入分享、完整明文、完整补偿项	加性分享、随机掩码、补偿分享
网络观察者	双方发送的 masked message	原始输入、TEE 内部随机量	x+a、y+b、w+c 等随机平移
REE	ECALL 参数、ECALL 返回值、本地线程缓冲	TEE 内部保存的全部随机状态	$r_0/r_1/r_2/\ldots$ 对中间值重新分享
TEE	被掩码的 ECALL 输入、TEE 内部随机量	两方完整输入分享集合	输入被随机拆分，TEE 只处理辅助项

理想情况下，任一单独视图都不能恢复完整明文。协议安全边界依赖两类假设：TEE 内部状态不会被非授权读取；双方不会恶意合谋恢复被保护输入。如果 CSP0 和 CSP1 合谋，它们本来就能把输入分享相加得到明文，这超出了二方秘密分享的隐私保护目标。

3.1.3 TEE 辅助与纯 MPC 的区别

纯 MPC 通常通过预处理三元组、OT 或同态技术生成乘法相关随机性。当前模块采用 TEE 辅助，思路是把一部分复杂随机相关性放入 Enclave 内部生成和保存。

这种设计的优点是协议结构更直接，工程实现更容易和现有 C++/SGX 框架结合。SMP 中的 $a/b/r_0$、3SMP 中的 $a/b/c/r_0/m_3$、ODReLU 中的 $a/b/r_0/r_1/v$ 都是 TEE 辅助随机状态的体现。

代价是安全假设包含 TEE 的可信性。也就是说，协议不只依赖密码学随机性，还依赖 Enclave 代码和运行环境对内部随机状态的保护。

3.1.4 随机种子的意义

实现中大量使用固定或派生种子，这些种子用于生成双方一致或可预测对应关系的随机数组。这样做可以减少显式传输随机数组的成本，也使测试结果可复现。

从理论角度看，协议需要的是独立随机量。工程测试中使用确定性伪随机生成器，是为了可复现实验；真实部署时，应当确保种子来源具有足够熵，并且不同会话、不同批次、不同协议调用之间避免随机量复用。

随机量复用会带来风险。例如同一组 a/b 被多次用于不同输入时，攻击者可能通过多次公开的 x+a 差分得到输入之间的关系：

```
(x + a) - (x' + a) = x - x'
```

因此，随机量的新鲜性是安全乘法和安全比较的重要前提。

3.2 核心协议理论

这一部分按协议类型展开理论重点：SMP 和 3SMP 解决环上乘法补偿，ODReLU 解决隐藏比较，ReLU 则说明如何把比较门值和乘法组合起来。

3.2.1 SMP 的理论细节

SMP 解决的是两方加性分享上的安全乘法问题。输入不是明文 x、y，而是两方分别持有的 $\langle x\rangle_0/\langle x\rangle_1$、$\langle y\rangle_0/\langle y\rangle_1$。协议完成后，双方也不应该得到明文乘积，而是得到 $\langle z\rangle_0/\langle z\rangle_1$，并满足：

```
z = z₀ + z₁ = x*y mod L
```

因此 SMP 的关键不是计算 D*E 本身，而是在不暴露 x、y、a、b 的情况下，把 D*E 中多出来的掩码影响全部扣掉。

公开 D、E 后，任意一方可以计算：

```
D*E = (x+a)(y+b)
    = xy + xb + ay + ab
```

目标是得到 xy，所以必须消除：

```
xb + ay + ab
```

等价写法为：

```
xy = D*E - D*b - E*a + a*b
```

这里的理论难点在于：D、E 可以公开，但 a、b 不能公开；D*b、E*a、a*b 又必须被正确加入最终结果。也就是说，协议需要让“知道随机掩码的一侧”帮助生成补偿，同时不能让 REE 或对方从补偿项中反推出随机掩码。

当前实现把补偿项拆成带随机数的分享。例如用 r_0 隐藏 TEE 返回结果，用 r_1/r_2 隐藏另一方分享参与补偿时的输入，使 TEE 看到的不是原始输入分享，REE 拿到的也不是完整补偿。最终每一方只得到一份输出分享：

```
CSP0: z₀
CSP1: z₁
z₀ + z₁ = D*E - D*b - E*a + a*b
```

从正确性角度看，只要公开主项和隐藏补偿项在环上相加后等于上式，SMP 就成立。从安全性角度看，单方看到的只有自己的输入分享、被随机平移后的公开值、以及被再次分享化的补偿结果，缺少恢复明文所需的另一方分享和完整随机掩码。

SMP 是后续协议的基础，因为安全 ReLU、Newton 迭代、近似函数修正等上层计算最终都需要在分享态下完成乘法。它的输出仍然是加性分享，这一点保证了协议可以继续串接，而不用在每一步打开中间值。

3.2.2 3SMP 的理论细节

3SMP 解决的是三输入乘法：

```
z = x*y*w mod L
```

它不是简单地把 SMP 重复调用两次，而是把三个输入的乘法放进一次更大的补偿结构中处理。这样做的目的，是减少中间结果同步和额外乘法轮次。

如果用两次 SMP 计算，流程通常是：

```
<t> = SMP(<x>, <y>)
<z> = SMP(<t>, <w>)
```

这个方式正确，但会引入两轮协议、两次网络同步、两次随机预处理和一次额外中间分享。对批量向量和神经网络算子而言，轮数往往比单次本地计算更影响整体延迟。3SMP 的价值就在于把 x*y*w 的展开一次性组织起来。

3SMP 的展开项更多，难点在于二阶交叉项：

```
abF, acE, bcD
```

这些项同时包含两个随机掩码和一个公开掩码值。如果逐项按 SMP 处理，会重新退化成多轮乘法。当前实现把二阶交叉项预先合并进 m_3，再用 m_4 处理一阶负补偿。

可以把 3SMP 看作三层结构：

```
第一层：公开主项 D*E*F
第二层：TEE 生成一阶负补偿 -aEF -bDF -cDE
第三层：TEE 生成二阶正补偿 abF + acE + bcD 和三阶修正 -abc
```

其中第一层可以由公开掩码值在 REE 侧计算；第二层需要 TEE 使用 a/b/c 生成负补偿；第三层需要 TEE 利用掩码之间的相关性生成二阶和三阶修正。m_3 承载二阶交叉项，m_4 承载一阶负补偿，r_0 再把最终补偿随机化成输出分享的一部分。

理论上，3SMP 的正确性来自完整展开式：

```
(D-a)(E-b)(F-c)
```

所有多出来的 a/b/c 相关项都必须被补偿掉，而且符号不能错。相比 SMP，3SMP 更容易出错的地方是二阶项和三阶项：漏掉 abF/acE/bcD 会使结果偏离输入相关的量，漏掉或符号写错 abc 会造成固定随机偏移。

3SMP 的工程价值在于减少组合乘法轮数。对高维向量而言，少一轮协议意味着少一次网络往返、少一次同步等待，也减少了上层算子流水线中的阻塞点。代价是单次协议内部状态更多，TEE 需要维护 a/b/c/r_0/m_3 等数组，并且实现中必须保证同一位置的 x[i]、y[i]、w[i] 和随机量严格对齐。

3.2.3 ODReLU 的理论细节

ODReLU 解决的是分享态下的符号判断问题。ReLU 的核心不是乘法，而是判断输入是否非负：

```
g = 1[x >= 0]
```

乘法协议可以在环上用代数补偿实现，但比较涉及数值顺序。顺序信息不能像加法、乘法那样直接在 mod L 环上公开计算，否则会泄露 x 的符号位。ODReLU 的目标是得到门值分享 <g>，而不是公开 g。

ODReLU 使用随机仿射变换把比较目标隐藏起来：

```
z = a*x + b
```

这里的 a 不仅随机，而且符号随机。若观察者不知道 a 的符号，就不能从变换后结果直接推出原始 x 的符号。TEE 保存符号位 v，并在第二步中把比较结果修正回对 x 的门值：

```
v = 0：变换保持方向
v = 1：变换翻转方向
```

因此外部即使参与了中间比较，也不能直接把中间结果解释成 x >= 0。只有 TEE 知道是否需要翻转门值。

b 的作用是扰动边界。没有 b 时，z = a*x 与 x = 0 的关系过于直接；加入 b 后，中间状态不再是单纯比例变换。实现中 abs_a、abs_b 由不同随机值构造，并保证二者不相等，使仿射变换具有更好的扰动性。

r_0 和 r_1 分别隐藏第一步和第二步 TEE 输出。第一步把输入分享和随机仿射参数组合起来，但返回值不能成为可解释的 a*x+b 明文；第二步把比较结果修正成门值分享，但返回值也不能成为明文 g。这两个随机量的作用就是让 REE 获得被重新分享的辅助结果，而不是可以直接解释的符号判定。

ODReLU 还依赖一个重要约定：输入 x 在 uint64_t 环中需要按有符号语义解释。也就是说，协议比较的不是普通无符号大小，而是固定点数或补码表示下的正负。实现和测试必须在这一点上保持一致，否则协议本身可能执行正确，但明文 reference 的解释不同，最终表现为边界样本错误。

从组合角度看，ODReLU 的输出 <g> 是一个门值分享。它本身不应该被打开，而应该直接作为 SMP 的输入继续参与乘法。这样符号信息只以分享形式存在，不会在 ReLU 过程中单独泄露。

3.2.4 ReLU 组合安全

ReLU 是组合协议，因此需要保证两个子协议之间不会破坏隐私。它把 ODReLU 产生的门值分享和原始输入分享结合起来：

组合过程如下：

```
1. ODReLU(<x>) -> <g>
2. SMP(<g>, <x>) -> <g*x>
3. 输出 <max(x, 0)>
```

正确性可以分两种情况理解：

条件	ODReLU 输出门值	SMP 计算	结果
x >= 0	<g> 重构为 1	<g> * <x>	<x>
x < 0	<g> 重构为 0	<g> * <x>	<0>

这里的关键是 <g> 不能被打开。如果打开 g，虽然不暴露 x 的精确值，但会暴露 x >= 0 这一位信息。对神经网络推理而言，大量 ReLU 激活模式本身可能泄露输入特征。因此正确做法是让 g 保持分享形式，并直接进入 SMP。

由于 SMP 对输入只要求加性分享，不要求输入来自原始明文还是来自另一个协议的输出，所以 ODReLU 的输出分享可以自然作为 SMP 的输入分享。这就是组合安全成立的工程基础。

ReLU 组合还要求两个子协议使用的新鲜随机量彼此独立。ODReLU 用于隐藏比较，SMP 用于隐藏门值和输入的乘法；如果随机量跨协议复用，可能把门值、输入和补偿项之间建立额外关联，削弱单个协议原本的安全边界。

从工程实现看，ReLU 的性能通常由两部分组成：

```
ReLU latency = ODReLU latency + SMP latency + synchronization overhead
```

其中 ODReLU 往往更受比较和 TEE step 影响，SMP 则更接近乘法补偿和通信开销。base 与 dualport 版本优化的是这两个子阶段之间或阶段内部的等待时间，而不是改变 ReLU(x) = g*x 这个组合关系。

因此验证 ReLU 时不能只看最终输出是否正确，还应分别关注 ODReLU 门值分享是否符合符号判断、SMP 是否正确消费 <g> 和 <x>、以及组合过程中是否意外打开了门值或中间乘积。只有这三点同时满足，ReLU 才同时具备正确性和隐私性。

3.3 工程调度边界

这一部分解释 base 和 dualport 只改变执行路径，不改变协议代数关系。只要线程同步和消息配对正确，工程优化不会改变协议正确性。

3.3.1 base 调度的理论边界

base 版本是普通单通道实现。它可以在一次协议调用内组织多个本地执行路径，例如：

```
线程 A: 构造 masked buffer 并发送
线程 B: 生成随机量并计算本地补偿
主线程: 发出启动信号，等待 join
```

这种并行不会改变任何数学等式。它只是把原本串行执行的步骤重叠起来：

```
串行:
prepare -> send -> compute -> join

base:
prepare
  ├─ send path
  └─ compute path
join
```

只要线程之间没有数据竞争，且最终使用的是同一批输入和同一批随机量，输出就与串行协议等价。

3.3.2 dualport 调度的理论边界

dualport 版本使用两个 BSNet 通道：

```
send_net
recv_net
```

这样做的核心是避免双方同时阻塞在同一个连接方向上。发送和接收可以拆开推进，协议数据也

可以按批次分成两部分，例如前半部分本地发送，后半部分等待对方消息后进行 TEE 或补偿合成。

从理论上看，dualport 改变的是消息传输路径，不改变消息内容和代数关系。因此 dualport 的正确性仍然由 SMP、3SMP、ODReLU、ReLU 的原始等式保证。

3.4 测试与性能评价

正确性测试关注：

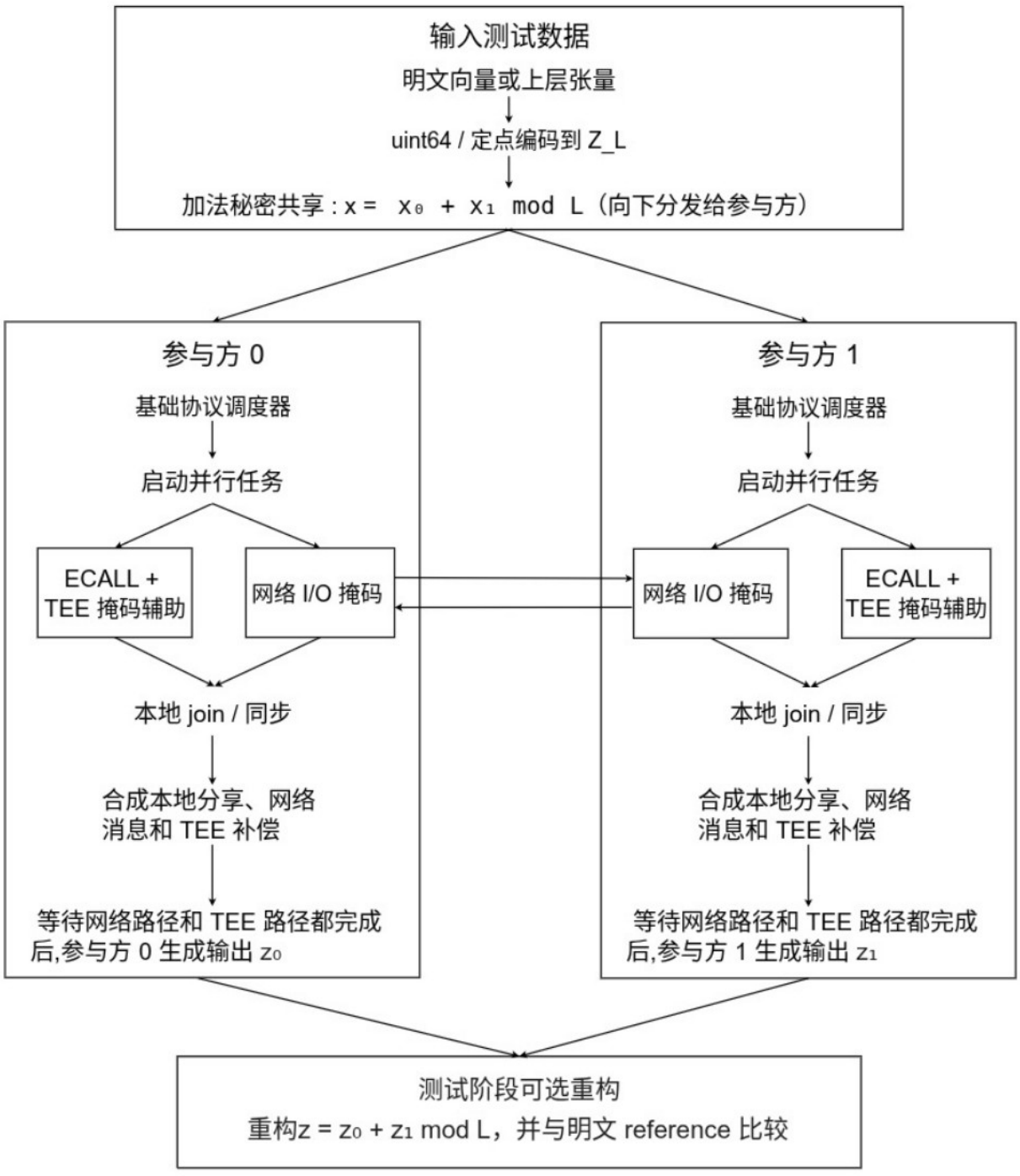
```
reconstructed_result == plaintext_reference
```

性能测试关注：

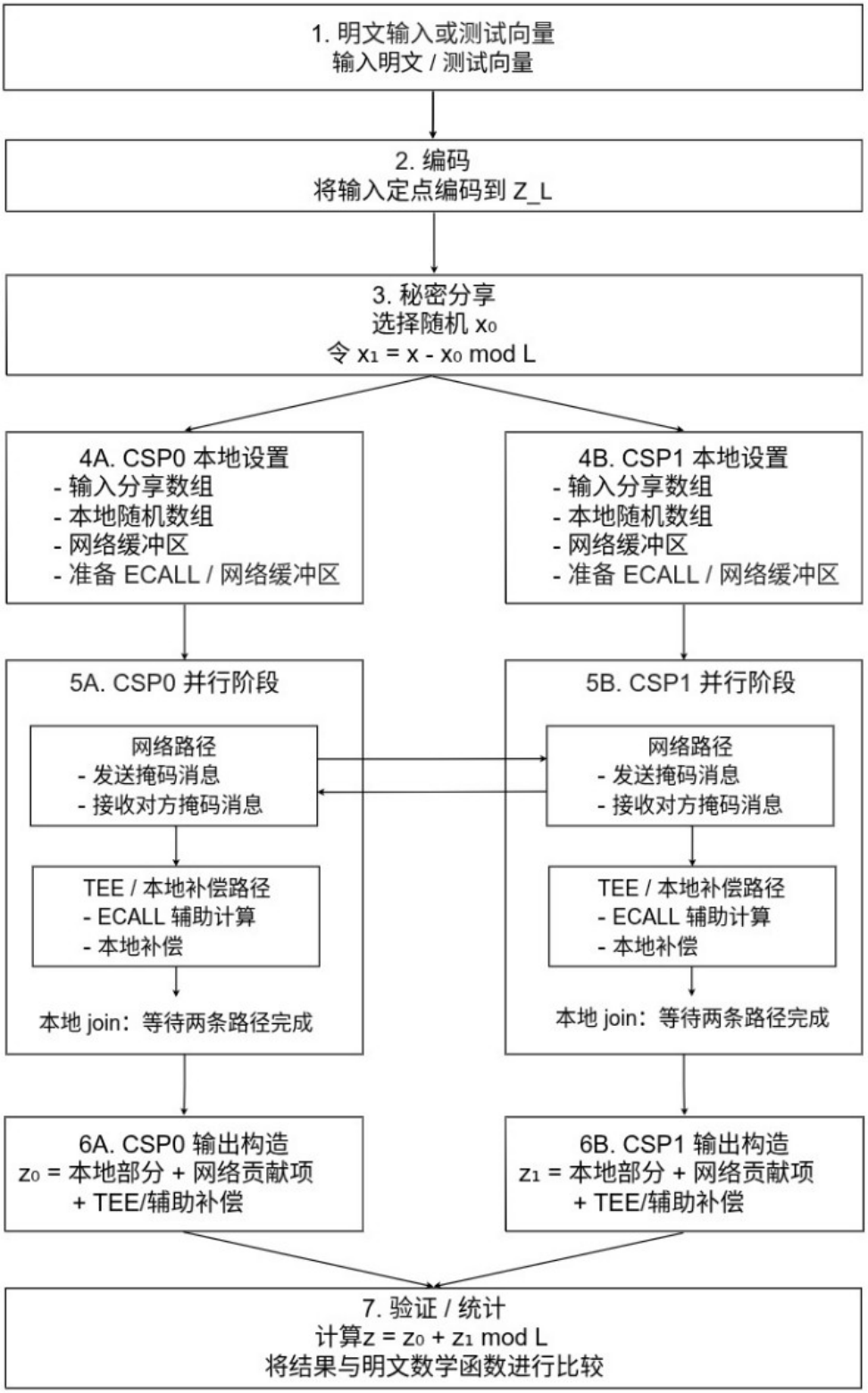
```
latency
communication bytes
base single-channel baseline
dualport blocking reduction
```

两者不能互相替代。一个协议可能计算正确但性能很差，也可能性能指标好但存在边界输入错误。因此基础协议模块需要同时保留准确性测试和计时版本。

4. 算法框架图


输入测试数据
明文向量或上层张量
uint64 / 定点编码到 Z_L
加法秘密共享 : x = x₀ + x₁ mod L（向下分发给参与方）
参与方 0
基础协议调度器
启动并行任务
ECALL + TEE 掩码辅助
网络 I/O 掩码
本地 join / 同步
合成本地分享、网络消息和 TEE 补偿
等待网络路径和 TEE 路径都完成后,参与方 0 生成输出 z₀
参与方 1
基础协议调度器
启动并行任务
网络 I/O 掩码
ECALL + TEE 掩码辅助
本地 join / 同步
合成本地分享、网络消息和 TEE 补偿
等待网络路径和 TEE 路径都完成后,参与方 1 生成输出 z₁
测试阶段可选重构
重构z = z₀ + z₁ mod L，并与明文 reference 比较

5. 数据流程图


1. 明文输入或测试向量
输入明文 / 测试向量
2. 编码
将输入定点编码到 Z_L
3. 秘密分享
选择随机 x0
令 x1 = x - x0 mod L
4A. CSP0 本地设置
- 输入分享数组
- 本地随机数组
- 网络缓冲区
- 准备 ECALL / 网络缓冲区
4B. CSP1 本地设置
- 输入分享数组
- 本地随机数组
- 网络缓冲区
- 准备 ECALL / 网络缓冲区
5A. CSP0 并行阶段
网络路径
- 发送掩码消息
- 接收对方掩码消息
TEE / 本地补偿路径
- ECALL 辅助计算
- 本地补偿
本地 join：等待两条路径完成
5B. CSP1 并行阶段
网络路径
- 发送掩码消息
- 接收对方掩码消息
TEE / 本地补偿路径
- ECALL 辅助计算
- 本地补偿
本地 join：等待两条路径完成
6A. CSP0 输出构造
z0 = 本地部分 + 网络贡献项
+ TEE/辅助补偿
6B. CSP1 输出构造
z1 = 本地部分 + 网络贡献项
+ TEE/辅助补偿
7. 验证 / 统计
计算z = z0 + z1 mod L
将结果与明文数学函数进行比较

6. 运行方式

在 party0 和 party1 目录分别执行：

```
./1.sh
```

测试宏由 App/define/define.h 控制。当前测试输出 GeLU、LayerNorm 和 Softmax 的多配置精度、耗时和通信统计。